github.com/mg-i-roc-maz/corsi · microservizi/lezione-4

LEZIONE 4 DI 8 · 3 ORE

Resilienza e configurazione

Quando catalog si blocca, orders non deve bloccarsi con lui. E deve dirci cosa sta succedendo.

Le tre ore di oggi

Teoria · 1h | **Laboratorio · 1h50**

Teoria

Fallimenti parziali, timeout, retry e idempotenza, circuit breaker, health check, configurazione, log e correlation ID.

Laboratorio

Blocchiamo catalog, poi rendiamo orders robusto: timeout, retry, circuit breaker con opossum, correlation ID.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se…?»

DA DOVE RIPARTIAMO

Lezione 3: con catalog spento, orders risponde 503. E se catalog fosse solo lentissimo?

SPENTO

Connessione rifiutata in un millisecondo. orders se ne accorge subito.

BLOCCATO

La connessione si apre, la risposta non arriva mai. orders aspetta.

INAFFIDABILE

Una risposta su tre è un errore. A volte va, a volte no.

FALLIMENTI PARZIALI

In un sistema distribuito qualcosa è sempre rotto.

Un monolite funziona o non funziona. Con tanti servizi, la domanda non è **se** qualcosa si guasta, ma **cosa fanno gli altri** quando succede.

Come può guastarsi catalog

Guasto	Cosa vede orders	Quanto è difficile
Spento	Connessione rifiutata, subito	Facile: errore immediato
Errori	Risposte 500 o 503	Facile: c'è un codice
Intermittente	A volte 200, a volte 503	Medio: un nuovo tentativo può bastare
Lento	La risposta arriva dopo 5, 30, 60 secondi	Difficile: sembra funzionare
Bloccato	La connessione resta aperta, nessuna risposta	Il più difficile: nessun errore, mai

Oggi li riproduciamo tutti: **docker compose stop**, **pause** e le variabili **CHAOS** di catalog.

Lento è peggio di spento

catalog spento

Ogni chiamata fallisce in **1 ms**. orders risponde 503 e passa alla richiesta successiva.

Le risorse di orders restano libere.

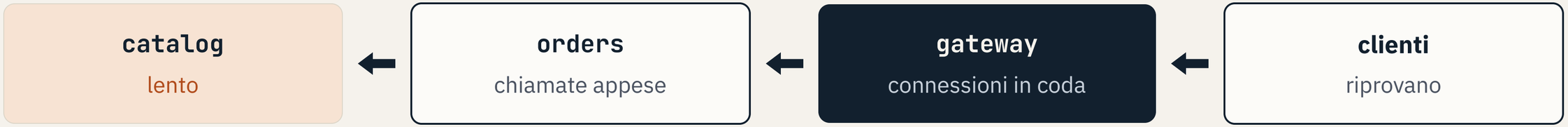
catalog bloccato

Ogni chiamata resta **aperta**. Ogni ordine tiene occupati una connessione, memoria, un posto in coda.

Le richieste si accumulano finché orders non riesce più a servire nessuno.

⚠ fetch di Node aspetta l'inizio della risposta fino a **5 minuti**, nginx 60 secondi. Senza un timeout scelto da noi, si aspetta il default di qualcun altro.

Il fallimento a cascata



1. catalog rallenta; orders aspetta e accumula richieste
2. orders esaurisce le risorse e rallenta anche lui
3. i clienti non ricevono risposta e **riprovano**: il carico raddoppia
4. alla fine cade tutto, anche le parti che non dipendevano da catalog

I pattern di stabilità

Michael Nygard, *Release It!*: una cassetta degli attrezzi contro i guasti a cascata.

Pattern	Idea	Nel corso
Timeout	Non aspettare per sempre	Oggi, Parte 2
Retry	Riprovare gli errori passeggeri	Oggi, Parte 3
Circuit breaker	Smettere di chiamare chi sta male	Oggi, Parte 4
Bulkhead	Limitare le risorse dedicate a una dipendenza	Extra 2
Fail fast	Se non puoi farcela, dillo subito	503 e Retry-After
Fallback	Una risposta di riserva	Extra 4, lezione 5

Il timeout: smettere di aspettare

```
const res = await fetch(url, {
  ...init,
  signal: AbortSignal.timeout(2000),
});
// dopo 2 s fetch lancia un errore
// con name === "TimeoutError"
```

Un timeout trasforma il guasto più difficile, **bloccato**, nel più facile: un **errore**.

Da lì in poi orders sa cosa fare: rispondere 503, riprovare, aprire il circuito.

Ogni chiamata di rete deve avere un timeout. Ogni. Senza eccezioni: anche verso il database, anche verso un servizio «sicuro».

Quanto deve durare un timeout?

Troppo **corto**: si interrompono richieste che sarebbero andate a buon fine.

Troppo **lungo**: non protegge, si aspetta comunque troppo.

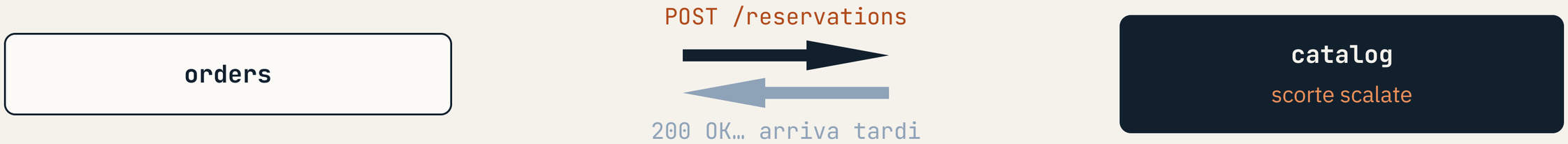
Regola pratica: poco sopra il tempo delle risposte più lente normali (il 99° percentile), misurato, non indovinato.

Il budget lungo la catena

Chi aspetta	Fino a
cliente → gateway	il browser decide
gateway → orders	10 s
orders → catalog	2 s × 3 tentativi
catalog → database	2 s

Chi chiama deve aspettare **più** di chi è chiamato, tentativi compresi.

Timeout non vuol dire «non è successo»



Il timeout scatta, orders pensa che sia fallita. Ma catalog **ha già scalato le scorte**: la risposta si è solo persa per strada.

Dopo un timeout lo stato è sconosciuto. Per una lettura non importa; per una scrittura sì.

Retry: ritentare può bastare

Aiuta

- un pacchetto perso in rete
- una copia del servizio che si sta riavviando
- un errore 503 passeggero

Il secondo tentativo, un attimo dopo, va a buon fine.

Peggiora

- un servizio sovraccarico: più richieste
- un servizio bloccato: tre volte l'attesa
- un errore 400 o 404: non cambierà mai
- un'operazione **non idempotente**

Si riprovano solo gli errori **passeggeri**, solo le operazioni **sicure da ripetere**, e non subito.

Idempotente: ripeterla non cambia il risultato

Metodo	Idempotente	Perché
GET	sì	Legge soltanto
PUT	sì	Sostituisce con lo stesso valore: due volte, stesso stato
DELETE	sì	Dopo la prima volta non c'è più niente da cancellare
POST	**no**	Ogni volta crea qualcosa di nuovo, o toglie altre scorte

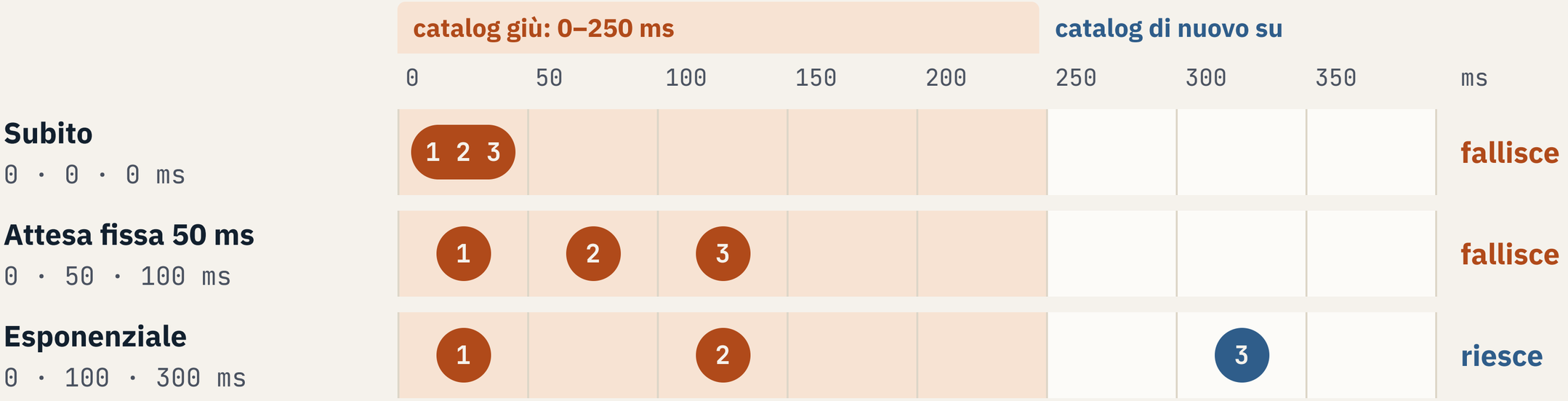
```
// getProduct: GET, si riprova
await withRetry(() ⇒ guarded(...), opts);

// reserve: POST, mai ripetuta
await guarded(.../reservations ...);
```

Ripetere una prenotazione dopo un timeout può togliere le scorte **due volte**. Per renderla sicura serve una **chiave di idempotenza**: lezione 8.

Backoff esponenziale: aspettare sempre di più

catalog si riavvia e per **250 ms** rifiuta le connessioni. orders fa **3 tentativi**: quando?



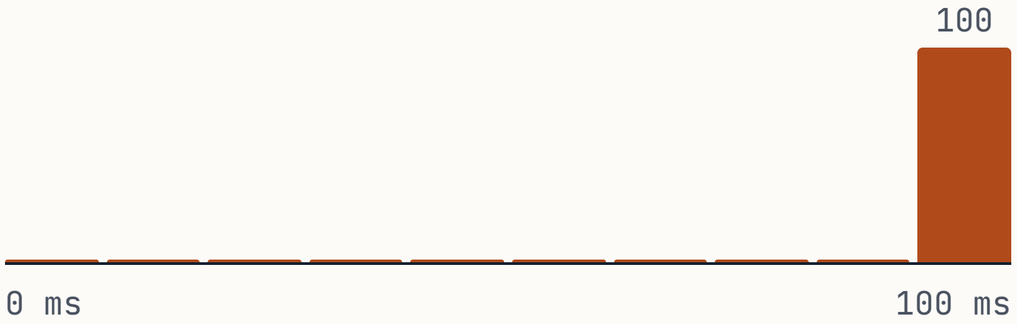

```
attesa n = base × 2^(n-1)
100 → 200 → 400 → 800 ms…
```

Stesse 3 chiamate, ma distribuite nel tempo: coprono un guasto più lungo e lasciano respirare catalog.

Il jitter: non riprovare tutti insieme

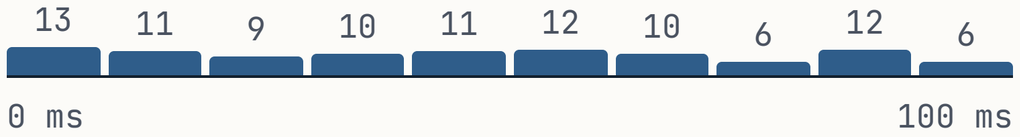
Esempio: **100 ordini** falliscono nello stesso istante, quando catalog si riavvia. Quando arrivano i loro secondi tentativi?

Senza jitter: tutti dopo 100 ms



Ogni barra: richieste in 10 ms. Qui 100 nello stesso millisecondo, su un catalog appena ripartito. Può ricadere, e il ciclo ricomincia.

Con jitter: tra 0 e 100 ms

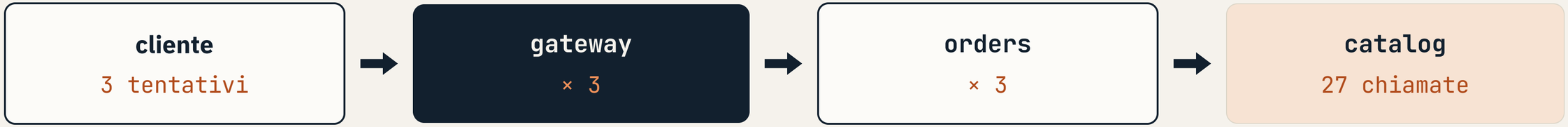


Le stesse 100 richieste spalmate: al massimo 13 ogni 10 ms. catalog le regge.

```
attesa = Math.random() × massimo
random 0,37 → 37 · 74 · 148 ms
```

Ogni client tira il suo numero: le attese crescono comunque, ma ognuno ha le sue.

I retry si moltiplicano



Se ogni livello riprova 3 volte, una sola richiesta del cliente diventa **27** chiamate a catalog, proprio quando catalog è in difficoltà.

Regola: si riprova in un solo punto della catena, di solito il più vicino al servizio che fallisce.

CIRCUIT BREAKER

Come l'interruttore di casa.

Quando un circuito elettrico va in cortocircuito, l'interruttore **scatta** e stacca la corrente prima che si bruci tutto. Poi qualcuno prova a riattaccare.

I tre stati del circuito



La prova riesce

Il circuito si **chiude**: tutto torna normale.

La prova fallisce

Il circuito si **riapre** per altri 10 secondi.

Il circuit breaker con opossum

```
import CircuitBreaker from "opossum";

const breaker = new CircuitBreaker(call, {
  timeout: false,
  errorThresholdPercentage: 50,
  volumeThreshold: 5,
  resetTimeout: 10000,
  rollingCountTimeout: 30000,
});

const res = await breaker.fire(path, init);
```

Opzione	Significato
`errorThresholdPercentage`	% di errori che apre
`volumeThreshold`	chiamate minime per giudicare
`resetTimeout`	dopo quanto provare
`rollingCountTimeout`	finestra in cui contare

Si chiama **breaker.fire(...)** al posto della funzione: stessi argomenti, stesso risultato.

Cosa conta come guasto?

Sì: colpa di catalog

- connessione rifiutata
- timeout
- risposte 500, 502, 503

No: risposte normali

- 404 prodotto inesistente
- 409 scorte insufficienti
- 400 dati non validi

⚠ Se un 409 contasse come guasto, cinque clienti che ordinano un prodotto esaurito **aprirebbero il circuito**, e nessuno potrebbe più ordinare niente.

Una storia vera: il circuito che non si apriva

Preparando questa lezione: catalog bloccato, 21 chiamate fallite di fila, circuito sempre **chiuso**. Perché?

Parametro	Valore
timeout di ogni chiamata	2 s
chiamate minime per giudicare	5
finestra di opossum (default)	10 s

La risposta

Una chiamata ogni 2 secondi: in 10 secondi la finestra contiene al massimo 4 o 5 chiamate, e le più vecchie escono. Non arriva mai a 5.

Soluzione: finestra di 30 s, BREAKER_WINDOW_MS.

Fallire in fretta, e dirlo bene

```
HTTP/1.1 503 Service Unavailable
Retry-After: 10
x-request-id: 5cbbc0ec...

{ "error": "Catalogo non disponibile,
            riprova più tardi",
  "reason": "circuit_open" }
```

Con il circuito aperto orders risponde in **3 ms** invece che in 6 secondi: niente risorse occupate, niente carico su catalog.

Retry-After dice al client quando ha senso riprovare.

Altre risposte di riserva (fallback)	Esempio
Un valore dalla cache	L'ultimo prezzo noto: extra 4, lezione 5
Un valore di default	«Disponibilità da verificare»
Funzione ridotta	Il catalogo si vede, gli ordini sono sospesi

Health check: vivo o pronto?

	Liveness: vivo?	**Readiness: pronto?**
Domanda	Il processo funziona?	Può servire richieste adesso?
Se no	Va **riavviato**	Niente traffico finché non torna pronto
catalog	`/health/live`: sempre ok	`/health/ready`: controlla il database
orders	`/health/live`: sempre ok	`/health/ready`: ok, con lo stato del circuito

⚠ Una liveness che controlla le dipendenze è pericolosa: se il database rallenta, tutti i servizi vengono **riavviati** insieme, e il riavvio non ripara il database.

Health check in Compose

```
catalog:
  healthcheck:
    test: ["CMD", "node", "-e",
      "fetch('http://localhost:3001/health/ready')
       .then(r ⇒ process.exit(r.ok ? 0 : 1))
       .catch(() ⇒ process.exit(1))"]
    interval: 5s
  restart: unless-stopped
```

Chiave	Effetto
healthcheck	stato healthy / unhealthy
depends_on: service_healthy	aspetta prima di partire
restart: unless-stopped	riavvia se il processo termina

Attenzione: Compose **non** riavvia un container unhealthy. Lo fa Kubernetes, con le sue probe.

Configurazione: fuori dal codice, controllata all'avvio

```
// services/orders/src/config.ts
timeoutMs: int(env, "CATALOG_TIMEOUT_MS", 2000, 1),
retries: int(env, "CATALOG_RETRIES", 2),
...
// CATALOG_TIMEOUT_MS=due →
// Configurazione non valida:
// CATALOG_TIMEOUT_MS=due (atteso un intero ⩾ 1)
```

- un solo file legge l'ambiente
- ogni valore ha un **default** sensato
- un valore sbagliato **blocca l'avvio** con un messaggio chiaro
- all'avvio si scrive nel log la configurazione usata

Fail fast vale anche qui: meglio un servizio che non parte, con un errore chiaro, che uno che parte e alle tre di notte usa un timeout di 0 ms.

Log strutturati

Testo libero

```
WARN retry p-1 attempt 2 after 187ms
```

Facile da leggere per una persona, difficile da filtrare per un programma.

JSON

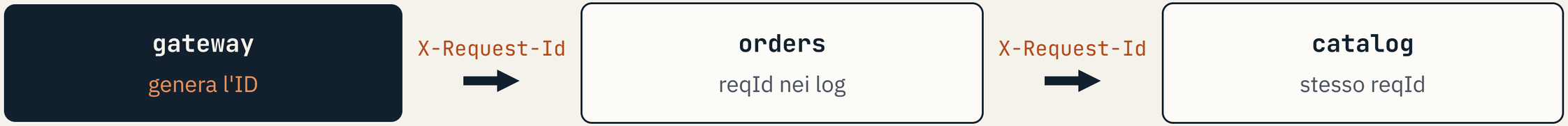
```
{"level":40,"reqId":"5cbb...",
 "productId":"p-1","attempt":2,
 "delayMs":187,
 "msg":"nuovo tentativo verso catalog"}
```

Ogni dato è un campo: si cerca, si conta, si filtra.

```
request.log.warn({ productId, attempt },
  "nuovo tentativo verso catalog");
```

Livelli: **debug**, **info**, **warn**, **error**. Si sceglie con LOG_LEVEL.

Il correlation ID




```
$ docker compose logs | grep 5cbbc0ec1e83a206d8d0afc07157e07c
gateway-1 | 5cbbc0ec... "POST /api/v1/orders" 201 → ...
orders-1 | {"reqId":"5cbbc0ec...","msg":"incoming request"...}
catalog-1 | {"reqId":"5cbbc0ec...","url":"/products/p-1"...}
catalog-1 | {"reqId":"5cbbc0ec...","url":"/products/p-1/reservations"...}
```

Un solo ID segue la richiesta in tutti i servizi: un **grep** ricostruisce la sua storia.

Log, metriche, tracce

	Cosa sono	Domanda a cui rispondono
Log	Eventi, uno per riga	Cosa è successo a questa richiesta?
Metriche	Numeri nel tempo: richieste al secondo, errori, latenza	Il sistema sta bene, adesso?
Tracce	Il percorso di una richiesta tra i servizi, con i tempi	Dove si perde il tempo?

Il correlation ID è la forma più semplice di traccia. Lo standard per farlo bene è **OpenTelemetry**: cenni nella lezione 8.

Verifichiamo: timeout e retry

1. Perché un servizio lento è peggio di uno spento?
2. Dopo un timeout su una POST, cosa sappiamo?
3. Perché getProduct si riprova e reserve no?
4. A cosa serve il jitter nel backoff?

Verifichiamo: circuito, salute, log

1. Nei tre stati del circuito, cosa succede a una chiamata?
2. Perché un 409 non deve aprire il circuito?
3. Perché la readiness di orders non controlla catalog?
4. Come trovi tutti i log di una richiesta?

Glossario

Termine	Significato
Fallimento parziale	Una parte del sistema è guasta, il resto funziona.
Fallimento a cascata	Un guasto che si propaga a chi dipende dal componente guasto.
Timeout	Tempo massimo di attesa di una risposta.
Idempotente	Un'operazione che ripetuta lascia lo stesso stato.
Backoff, jitter	Attese crescenti tra i tentativi, con una parte casuale.
Circuit breaker	Interrompe le chiamate verso un servizio che fallisce troppo.
Liveness, readiness	Il servizio è vivo? È pronto a ricevere traffico?
Correlation ID	Identificativo che segue una richiesta in tutti i servizi.

Laboratorio

15'

1 · L'esperimento

Blocchiamo catalog e guardiamo.

20'

2 · Timeout

TODO 1: non aspettare per sempre.

20'

3 · Retry

TODO 2 e 3: riprovare le letture.

25'

4 · Circuit breaker

TODO 4: opossum, poi di nuovo l'esperimento.

15'

5 · Correlation ID

TODO 5: un ID in tutti i log.

```
cd microservizi/lezione-4/start → docker compose up --build -d
```

Il repo della lezione

`github.com/mg-i-roc-maz/corsi`

```
cd corsi
git pull
cd microservizi/lezione-3/start
docker compose down
cd ../../lezione-4/start
docker compose up --build -d
docker compose ps
```

microservizi/lezione-4/

Cartella	Contenuto
`slides.pdf`	Queste slide, in PDF
`start/`	**Si lavora qui**
`solution/`	La soluzione, per riallinearsi
`extra/`	Esercizi extra, test e soluzioni

Dopo qualche secondo **docker compose ps** deve mostrare quattro servizi **healthy**.

Cosa c'è in start

gateway/nginx.conf	**TODO 5 · timeout già pronti**
services/catalog/	
src/chaos.ts	lento o inaffidabile
src/app.ts	/health/live, /health/ready
services/orders/	
src/config.ts	variabili, controllate
src/catalog-client.ts	**TODO 1, 3, 4**
src/retry.ts	**TODO 2**
compose.yaml	healthcheck, restart, CHAOS

È la soluzione della lezione 3, più tutto quello che serve per gli esperimenti.

Nuova dipendenza in orders: **opossum**.

```
cd services/orders
npm install
```

1.1 · Blocca catalog

```
docker compose logs -f orders # 2° terminale

docker compose pause catalog
# ora: POST /api/v1/orders da shop.http
# e, mentre aspetti, GET /api/v1/products
docker compose unpause catalog
```

1. Quanto aspetti? Cosa ricevi alla fine?
2. La lettura dei prodotti risponde?
3. Dopo unpause, guarda GET /api/v1/orders: c'è un ordine in più?

1.2 · Bloccato o spento?

```
docker compose stop catalog
# POST /api/v1/orders
docker compose start catalog

CHAOS_DELAY_MS=3000 docker compose up -d catalog
# POST /api/v1/orders
docker compose up -d catalog
```

catalog	orders risponde
fermato	503, subito
in pausa	niente: il gateway rinuncia dopo 10 s
lento, 3 s	201, ma dopo almeno 6 s

Perché 6 secondi con un ritardo di 3? Quante chiamate fa orders a catalog per un ordine?

2.1 · I test di oggi

```
cd services/orders
npm install
npm test
```

Nessun Docker: un piccolo server finto fa la parte di catalog lento, rotto o sano.

ALL'INIZIO

```
# pass 25 · # fail 6
```

OBIETTIVO

```
# pass 31 · # fail 0
```

2.2 · TODO 1: il timeout

In **call()**, dentro src/catalog-client.ts:

1. passa a fetch **signal: AbortSignal.timeout(timeoutMs)**
2. nel catch, se **err.name === "TimeoutError"**, lancia CatalogUnavailableError con kind **"timeout"**

Verifica

```
npm test
```

Passa «timeout: un catalog lento viene abbandonato dopo timeoutMs».

3.1 · TODO 2: withRetry

```
for (let attempt = 1; ; attempt++) {
  try {
    return await operation();
  } catch (err) {
    // finiti i tentativi, o errore da non
    // ripetere? rilancia
    // altrimenti: calcola l'attesa,
    // avvisa con onRetry, aspetta
  }
}
```

Il ciclo è il cuore del retry. **backoffDelay** è già scritta: calcola l'attesa con backoff esponenziale e jitter.

```
await new Promise(
  (r) ⇒ setTimeout(r, delay));
```

Quante volte viene chiamata operation() con retries = 2?

3.2 · TODO 3: retry sulle letture

```
const res = await withRetry(
  () ⇒ guarded(`/products/${...}`,
                { method: "GET" }, ctx),
  { retries, baseMs: retryBaseMs,
    shouldRetry: retryable,
    onRetry: (err, attempt, delayMs) ⇒
      log?.warn({ productId: id, attempt,
                  delayMs }, "nuovo tentativo") },
);
```

⚠ Non toccare **reserve**: una prenotazione non si ripete. Il test «una prenotazione (POST) NON viene mai ripetuta» lo controlla.

Con i TODO 1–3: **npm test** deve fallire solo sul circuit breaker.

4.1 · TODO 4: il circuit breaker

1. crea **new CircuitBreaker(call, { ... })** con i valori di breakerOptions
2. in **guarded()** usa **breaker.fire(path, init, ctx)**
3. se fallisce a circuito aperto: CatalogUnavailableError con kind **"circuit_open"**
4. **circuitState()**: open, half-open o closed

Utile

```
breaker.opened
breaker.halfOpen
breaker.on("open", ...)
```

I commenti del TODO contengono lo scheletro delle opzioni.

4.2 · Di nuovo l'esperimento

```
docker compose up --build -d
docker compose pause catalog
# quattro ordini di fila da shop.http
# GET http://localhost:8080/health/orders
docker compose unpause catalog
# un ordine subito, uno dopo 10 s
```

Ordine	Tempo	reason
1°	~6 s	timeout
2°	~4 s	circuit_open
3° e 4°	pochi ms	circuit_open
dopo unpause	pochi ms	circuit_open
dopo 10 s	201	—

5.1 · TODO 5: il correlation ID nel gateway

```
log_format gateway '$request_id $remote_addr ...';

proxy_set_header X-Request-Id $request_id;
```

```
docker compose restart gateway
```

orders e catalog sono già pronti: usano l'header **x-request-id** se arriva, e lo scrivono in ogni riga di log come **reqId**.

orders lo passa anche a catalog in ogni chiamata.

5.2 · Segui una richiesta

1. crea un ordine da shop.http
2. copia l'header **x-request-id** della risposta
3. cercalo in tutti i log

```
docker compose logs | grep <id>
```

Poi prova

```
LOG_LEVEL=warn docker compose up -d
LOG_LEVEL=debug docker compose up -d
```

Quali righe spariscono con warn? Quali compaiono con debug?

Un catalog inaffidabile

```
CHAOS_FAIL_PERCENT=30 docker compose up -d catalog
```

Ora venti ordini e venti letture di **GET /api/v1/products/p-2**. Quanti falliscono di ciascuno?

	Falliscono
Letture dirette	circa 3 su 10: il gateway non riprova
Ordini	circa 3 su 10, ma per la POST: la GET viene riprovata

Il retry nasconde gli errori delle letture, non quelli delle prenotazioni.

Checkpoint

Verifica	Esito atteso
`npm test in services/orders`	pass 31, fail 0
`docker compose ps`	quattro servizi healthy
`ordine con catalog in pausa`	503 dopo circa 6 s, reason timeout
`ordini successivi`	503 in pochi ms, circuit_open, Retry-After
`GET /health/orders`	circuit open, poi closed dopo unpause
`docker compose logs | grep`	lo stesso ID in gateway, orders e catalog

Consegna: l'output di npm test e le righe trovate con grep per un ordine.

Se qualcosa non va

Sintomo	Causa e rimedio
`Con il circuito aperto orders risponde 500`	L'errore di opossum non è tradotto in CatalogUnavailableError
`Il circuito non si apre mai`	rollingCountTimeout mancante: la finestra di 10 s è troppo corta
`Il circuito si apre con prodotti esauriti`	404 o 409 trattati come errori dentro call()
`Dopo unpause gli ordini falliscono ancora`	Normale per 10 s: il circuito è aperto
`orders non parte: Configurazione non valida`	Una variabile con un valore sbagliato: leggi il log
`Il chaos resta attivo`	docker compose up -d catalog senza variabili
`Tutto risponde 502 dopo una pause`	docker compose unpause catalog

ULTIMI 10 MINUTI

Cosa si rompe se...?

...il cliente vede un errore dopo 10 s e riprova, ma orders stava ancora lavorando?

...orders gira in tre copie? Quanti circuiti ci sono, e sono d'accordo?

...il database di catalog diventa lento, ma non si ferma?

PROSSIMA LEZIONE

Lezione 5 · Caching: fondamenti

Redis nel Compose e cache-aside su catalog: misuriamo i tempi con e senza cache, e scopriamo perché invalidare è difficile.

Per la volta prossima: npm test verde in orders e l'esperimento del circuito riuscito.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	I numeri del circuito	base	orders
2	Una paratia tra orders e catalog	medio	orders
3	Retry nel gateway	medio	nginx.conf
4	Fallback con l'ultimo prezzo noto	avanzato	orders
5	Il budget dei tempi	carta e penna	—

Testi in lezione-4/extra/README.md. Per 1, 2 e 4 ci sono test pronti; le soluzioni sono in lezione-4/extra/soluzione.

I numeri del circuito

Aggiungi a **/health/ready** di orders i contatori del circuito.

```
"dependencies": { "catalog": {
  "circuit": "open",
  "stats": { "fires": 12, "failures": 5,
    "timeouts": 0, "rejects": 7,
    "successes": 0 } } }
```

Domanda

Con il circuito aperto, quale contatore cresce? Perché i **timeouts** di opossum restano a zero?

Una paratia tra orders e catalog

Al massimo **N chiamate contemporanee** verso catalog; le altre rifiutate subito.

- opossum: opzione **capacity**
- oltre il limite: errore con **code "ESEMLOCKED"**
- traducilo in kind **"too_busy"**, senza retry
- configurabile: **CATALOG_MAX_CONCURRENT**

Domanda

Meglio rifiutare subito o mettere in coda? Cosa succede a una coda se catalog resta lento per minuti?

Retry nel gateway

Con due copie di catalog, se una risponde 503 il gateway prova l'altra.

```
proxy_next_upstream error timeout
                    http_502 http_503;
proxy_next_upstream_tries 2;
```

```
CHAOS_FAIL_PERCENT=50 docker compose up -d \
  --scale catalog=2
docker compose restart gateway
```

Domanda

nginx ripete anche i POST su un'altra copia? Perché è giusto così?

Fallback con l'ultimo prezzo noto

Se catalog non risponde, usa l'ultimo prodotto letto, se abbastanza recente.

- una **Map**: id → prodotto e ora di lettura
- restituiscilo con **stale: true**
- età massima: **CATALOG_FALLBACK_MAX_AGE_MS**, 0 = spento

Prima di scrivere codice

Il prezzo può essere cambiato. Chi perde, il negozio o il cliente? E le scorte passano comunque da catalog: serve davvero, qui?

Il budget dei tempi

Timeout 2 s, 2 tentativi in più, circuito a 5 chiamate. Il gateway aspetta orders 10 s.

1. catalog bloccato, ordine di **tre** prodotti: quanto aspetta orders, al massimo?
2. Cosa riceve il cliente oltre i 10 s? orders si ferma?
3. Il cliente riprova. Cosa succede all'ordine?

Infine

Proponi due modifiche per stare dentro il budget.

Per approfondire

M. Nygard · *Release It!*	Pragmatic Bookshelf, 2ª ed. 2018: i pattern di stabilità
M. Fowler · *CircuitBreaker*	martinfowler.com/bliki: il pattern in una pagina
AWS Architecture Blog · *Exponential Backoff And Jitter*	Perché e come aggiungere il jitter
opossum · documentazione	nodeshift.dev/opossum: opzioni, eventi, statistiche
Kubernetes Docs · *Liveness, Readiness and Startup Probes*	Le probe e quando usarle
OpenTelemetry · *Observability primer*	opentelemetry.io: log, metriche e tracce